PRAKTIKUM
SISTEM BASIS DATA

Nama	**Valiant Joshua**	**No. Modul**	**8**
NPM	**2406352153**	**Tipe**	**Case STudy**

1. Controller user/email

CS_VR_ValiantJoshua_2406352153_SBD6 > backend > src > database > redis.js > ...

```js
const Redis = require("ioredis");

const redis = new Redis();

new Redis({
  port: 6379, // Redis port
  host: "127.0.0.1", // Redis host
});

redis.on("connect", () =>{
    console.log("Connected to redis's server");
});

module.exports = redis;
```

2. Implementasi cache:
   - Perubahan pada user controller:

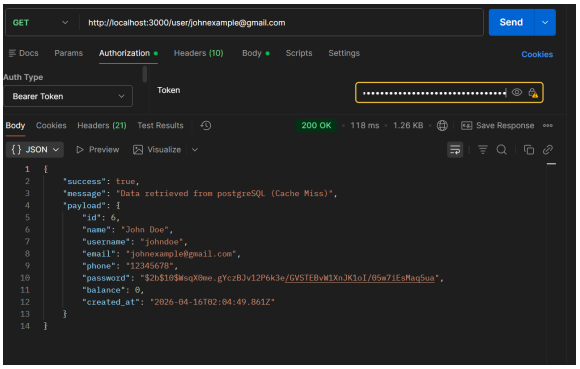
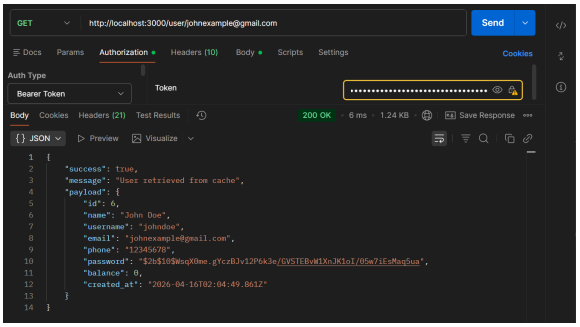
```js
static async getUserByEmail(req, res){
    try{
      const {email} = req.params;
      const redisKey = `user:${email}`;
      const cacheedData = await redis.get(redisKey);
```

```
      if (cacheedData){
        return res.status(200).json({
          success: true,
          message: "User retrieved from cache",
          payload: JSON.parse(cacheedData)
        })
      }

      const user = await UserService.getUserByEmail(email);

      if (!user){
        return res.status(400).json({
          success: false,
          message: 'User not found'
        })
      }
      await redis.set(redisKey, JSON.stringify(user), 'EX', 60)
      res.status(200).json({
        success : true,
        message: "Data retrieved from postgreSQL (Cache Miss)",
        payload : user
      })
    } catch(err){
      next(err);
    }
  }
}
```

- SS Postman:

Get Pertama (dari Postgres) Response time: 118 ms	
Get Kedua (dari cache) Response time: 6ms	

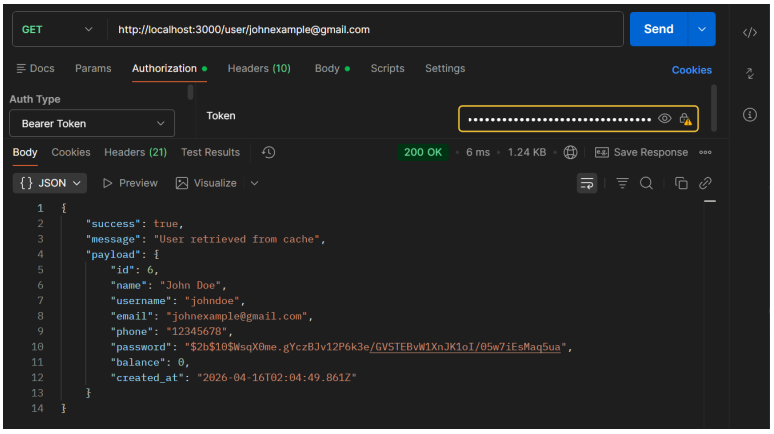
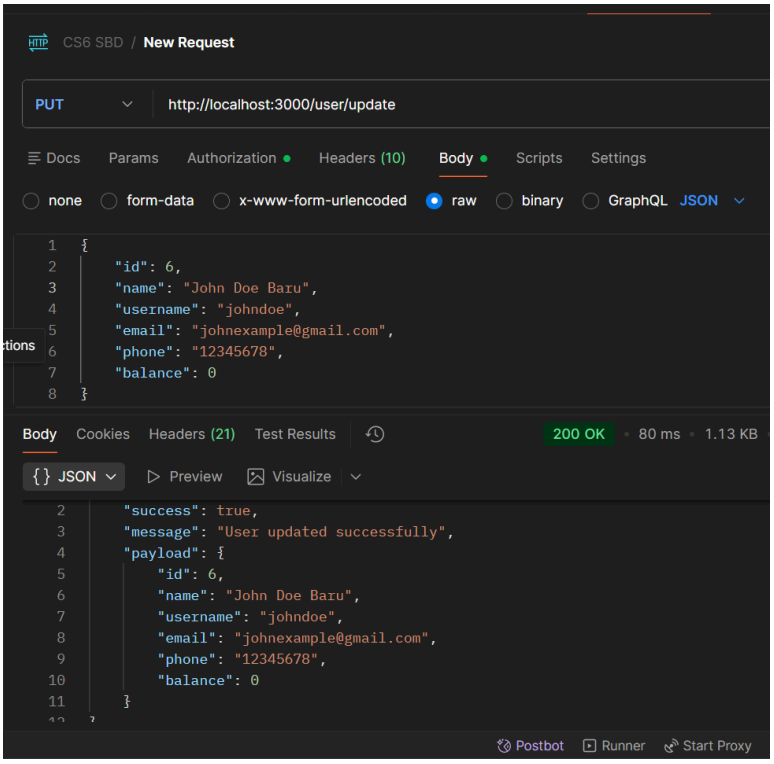
3. Implementasi cache invalidation pada updatedProfile:

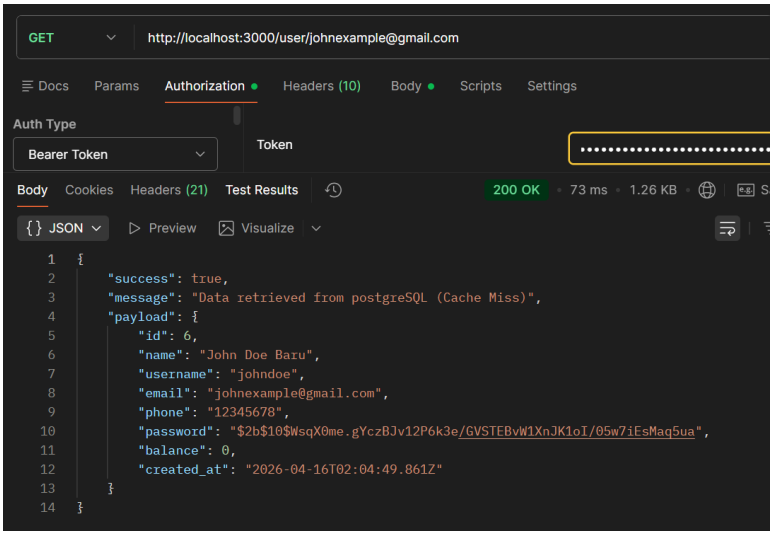
```
  static async updateProfile(req, res, next) {
    try {
      const { id, name, username, email, phone, password, balance } =
req.body;
      const updatedUser = await UserService.updateProfile(id, { name,
username, email, phone, password, balance });

      if (updatedUser && updatedUser.email){
        const redisKey = `user:${updatedUser.email}`;
        await redis.del(redisKey);
      }
      res.status(200).json({
        success: true,
        message: 'User updated successfully',
        payload: updatedUser,
      });
    } catch (error) {
      next(error);
```

```
    }
  }
```

SS Postman:

Pertama saya melakukan get 2-3 kali hingga mengambil data dari cache	
Lalu kemudian langsung melakukan update/put	

Kemudian melakukan get lagi. Dari get ini dapat dilihat bahwa data diambil dari postgres/ cache miss, karena pada Update, data dihapus dari redis, sehingga harus mengambil ulang dari database	

4. /transaction/create:

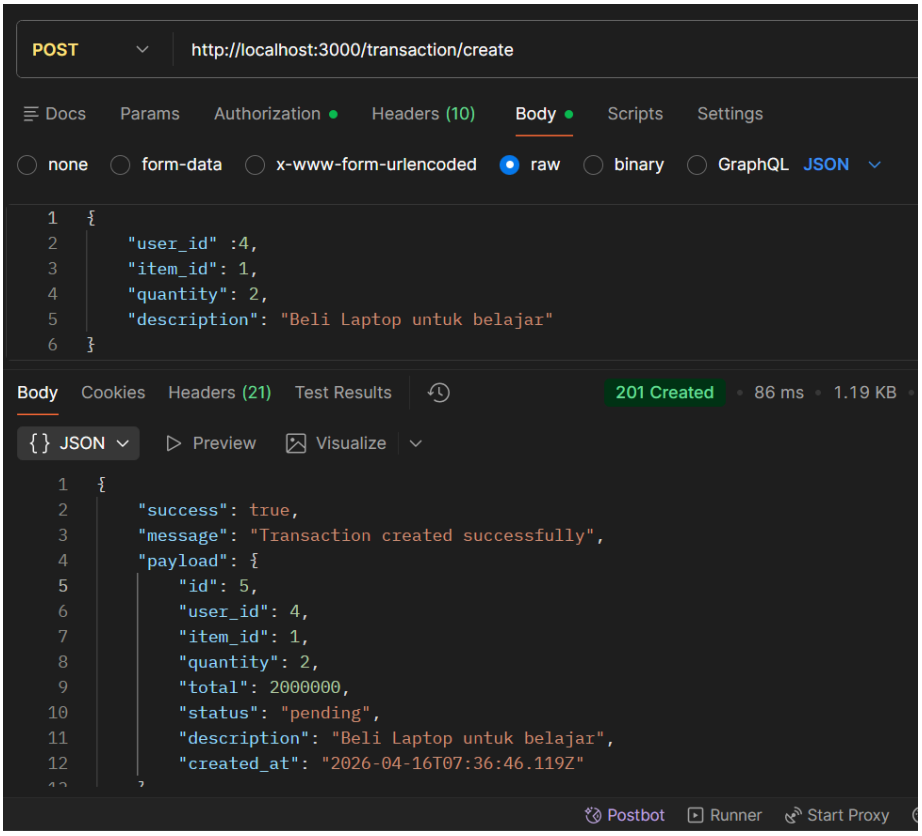
```
  static async createTransaction(req, res, next) {
    try {
      const { user_id, item_id, quantity, description } = req.body;
      const transaction = await TransactionService.createTransaction({
user_id, item_id, quantity, description });

      const totalTransaksi = transaction.total || 0;

      const messageId = await redis.xadd(
        'transaction-logs',
        '*',
        'userId', user_id,
        'itemId', item_id,
        'total', totalTransaksi
      );
      console.log(`[Redis Stream] Transaction berhasil dicatat ke stream
transaction-logs dengan messageID  : ${messageId}`);
      res.status(201).json({
        success: true,
        message: 'Transaction created successfully',
        payload: transaction,
      });
    } catch (error) {
```

```
        next(error);
    }
  }
```

SS Postman:




```
Connected to PostgreSQL database
[Redis Stream] Transaction berhasil dicatat ke stream transaction-logs dengan messageID  : 1776325006127-0
```

5. Kesimpulan:

   Berdasarkan percobaan di nomor 2 (melakukan get user by email), waktu yang dibutuhkan jika menggunakan redis (mengambil dari cache) adalah 6 ms, sedangkan jika diambil dari postgre (cache miss) secara langsung, waktu yang dibutuhkan adalah 118ms jauh lebih lama dibanding dengan menggunakan redis.

   Hal ini terjadi karena Redis merupakan sistem penyimpanan in memory (cache disimpan di RAM) dan menggunakan struktur key value yang sederhana, jadi hanya perlu mencocokan string key tersebut dan langsung menemukanletak nilainya di memory, jadi kompleksitanya 0(1). Sedangkan

postgreSQL yang merupakan disk based database. Arsitektur dari disk based database sendiri di design untuk membaca dan menulis yang melibatkan operasi input output mekanis yang memakan waktu jauh lebih lama, juga karena harus melakukan beberapa step seperti memparsing sql, query planner, dll sehingga kompleksitas waktunya 0(log n)